Multi-Agent Drone Routing in a Temporal Urban Grid

1. Background

In the year 2026, urban logistics have fully transitioned to autonomous drone fleets. However, city regulations have grown increasingly complex. No-Fly Zones (NFZs) are now dynamic areas (e.g., around scheduled stadium events, VIP convoys, or construction sites) that activate and deactivate at specific times.

Your objective is to build a routing engine that manages a fleet of drones to successfully deliver packages while navigating these shifting constraints and optimizing limited battery life.

2. Problem Statement

You are provided with a 2D coordinate grid representing a city. You must develop a system that assigns and routes a fleet of **N** drones from a central warehouse to complete **M** deliveries.

- Drones start at the warehouse, located at the center of the map: `(map_size[0] / 2, map_size[1] / 2)`
- Each drone can carry multiple packages in a single trip, provided the total weight does not exceed the drone's `max_payload` capacity
- After completing deliveries, a drone must return to the warehouse or a charging station; mid-air stops are prohibited
- All distances are Euclidean (straight-line)
- Drones fly in straight lines between consecutive path points
- Drones do not collide with one another; multiple drones may occupy the same coordinates simultaneously

3. Constants (Fixed for All Test Cases)

- **Drone speed:** 1 distance unit per timestep
- **Battery capacity:** 500 energy units (per drone, fully charged at start)
- **Charge rate:** 2 energy units per timestep (at charging stations)

Input Format

Your program must accept a JSON file with the following structure:

```
{
  "map_size": [Width, Height],
  "drones": [
    {"id": "drone_1", "max_payload": 1.0},
```

```
    {"id": "drone_2", "max_payload": 0.8}
  ],
  "deliveries": [
    {"id": "d1", "x": 20, "y": 30, "weight": 0.3, "deadline": 200}
  ],
  "charging_stations": [
    {"x": 50, "y": 50, "slots": 2}
  ],
  "no_fly_zones": [
    {
      "shape": "circle",
      "center": [80, 80],
      "radius": 15,
      "T_start": 0,
      "T_end": 150
    },
    {
      "shape": "rectangle",
      "corners": [[120, 120], [140, 160]],
      "T_start": 50,
      "T_end": 300
    }
  ]
}
```

Field Definitions

- **map_size:** [Width, Height] of the grid. Warehouse is located at `(Width/2, Height/2)`
- **drones:** List of drones. Each drone includes:
  - `id`: Unique identifier
  - `max_payload`: Maximum carrying capacity
- **deliveries:** List of delivery requests. Each includes:
  - `id`: Delivery identifier
  - `x, y`: Coordinates of delivery location
  - `weight`: Package weight
  - `deadline`: Latest allowable delivery time
- **charging_stations:** List of charging stations:
  - `x, y`: Location of the station
  - `slots`: Number of drones that can charge simultaneously
- **no_fly_zones:** List of NFZs:
  - **Circle:**
    - `center`: [x, y]
    - `radius`
  - **Rectangle:**
    - `corners`: [[x_min, y_min], [x_max, y_max]]
  - `T_start` and `T_end`: Active time window

Constraints

1 No-Fly Zones (NFZs)

- NFZs are defined as circular or rectangular areas with a specific time window `[T_start, T_end]`
- A drone cannot enter or pass through an NFZ while it is active
- NFZs are known upfront; there are no surprise obstacles
- A drone may pass through the same area when the NFZ is inactive

NFZ Collision During Transit:

Drones move in straight lines between consecutive path points at a speed of 1. A segment of length `d` takes `d` timesteps to traverse.

If a drone departs point **A** at time `t0` and travels toward point **B** (distance `d`), it reaches any intermediate point **P** at time:

```
t0 + dist(A, P)
```

- If an NFZ is active at point **P** at that time, the path is blocked
- If the NFZ deactivates before arrival, the drone may pass through

Waiting Strategy:

- If a path is blocked, the drone must wait at its current location
- No battery is consumed while waiting
- After deactivation, the drone resumes its path

Routing Options:

- Detour around active NFZs using waypoints
- Wait safely for the NFZ to deactivate

2 Energy Model

Energy consumed per leg is calculated as:

```
E_leg = distance × (1 + current_payload_weight)
```

- `current_payload_weight` = total weight of packages on the drone
- Payload decreases after deliveries, reducing energy cost
- Battery levels must always remain ≥ 0
- If a drone cannot complete a trip and return safely, it should not attempt it

3 Charging Stations

- Charging stations have limited slots
- If all slots are occupied, drones must wait (no battery usage)
- Charging rate is 2 energy units per timestep
- Drones may leave once sufficient charge is reached (full charge not required)

4 Deliveries & Deadlines

- Each delivery has a strict deadline
- Arrival must be on or before the deadline
- Missed deadlines result in failure (no partial credit)
- Packages are picked up at the warehouse and delivered in the field

5 Warehouse & Pickup Rules

- **PICKUP** actions only occur at the warehouse
- Upon return, drones are fully recharged to 500 energy units
- Drones can make multiple trips:

```
Warehouse → Recharge → Pickup → Deliver → Repeat
```

6 Multi-Package Trips

- Drones can carry multiple packages if total weight ≤ `max_payload`
- Example route:

```
Warehouse → Delivery A → Delivery B → ... → Warehouse / Charging Station
```

- Delivery order matters:
  - Dropping heavier packages first reduces energy costs for later legs

Output Format

Your program must produce a **Flight Manifest** as a JSON file:

```
{
  "flight_manifest": [
    {
      "drone_id": "drone_1",
      "path": [
        {
          "x": 50,
          "y": 50,
          "t": 0.0,
          "action": "PICKUP",
          "delivery_ids": ["d1", "d3"]
        },
        {
          "x": 20,
          "y": 30,
```

```
          "t": 42.4,
          "action": "DELIVER",
          "delivery_id": "d1"
        },
        {
          "x": 35,
          "y": 60,
          "t": 75.1,
          "action": "DELIVER",
          "delivery_id": "d3"
        },
        {
          "x": 50,
          "y": 50,
          "t": 92.0,
          "action": "RETURN"
        }
      ]
    }
  ]
}
```

Action Types

- **PICKUP:** Drone picks up packages at the warehouse. Must include `delivery_ids` (list).
- **DELIVER:** Drone delivers a package. Must include `delivery_id`.
- **CHARGE:** Drone arrives at a charging station to recharge.
- **CHARGE_COMPLETE:** Drone finishes charging and departs.
- **WAIT:** Drone waits at its current position for an NFZ to deactivate.
- **WAYPOINT:** Intermediate navigation point.
- **RETURN:** Drone returns to warehouse or charging station (end of trip).

Scoring

Your solution is scored using a custom checker. The score formula is:

```
raw_score = (successful_deliveries × 100) − (total_energy × 0.1) − (makespan × 0.05)
```

Where:

- **successful_deliveries:** Number of deliveries completed on time (arrived at destination ≤ deadline) without any constraint violations.
- **total_energy:** Sum of energy consumed across all drone legs. Energy per leg = `distance × (1 + current_payload_weight)`.
- **makespan:** The latest timestamp in the entire flight manifest (i.e., when the last drone finishes).

Your score per test case is the raw_score. Higher is better. Invalid solutions (constraint violations) score 0. The leaderboard ranks by total score across all test cases.

Validation Rules (violations → score 0)

- A drone's path must start with PICKUP and end with RETURN.

- Time must be monotonically non-decreasing along each drone's path.
- Travel time between consecutive points must equal `distance / speed` (speed = 1).
- Battery must never go below 0.
- Payload weight at any point must not exceed `max_payload`.
- No drone path segment may pass through an active NFZ.
- A delivery is only counted if the drone arrives at the exact delivery coordinates on or before the deadline.
- Each delivery can only be delivered once.

Sample Input 0

```
{
  "map_size": [100, 100],
  "drones": [
    {"id": "drone_1", "max_payload": 1.0}
  ],
  "deliveries": [
    {"id": "d1", "x": 70, "y": 60, "weight": 0.3, "deadline": 200},
    {"id": "d2", "x": 30, "y": 80, "weight": 0.4, "deadline": 200}
  ],
  "charging_stations": [],
  "no_fly_zones": []
}
```

Sample Output 0

```
{
  "flight_manifest": [
    {
      "drone_id": "drone_1",
      "path": [
        {
          "x": 50,
          "y": 50,
          "t": 0.0,
          "action": "PICKUP",
          "delivery_ids": [
            "d1",
            "d2"
          ]
        },
        {
          "x": 70,
          "y": 60,
          "t": 22.36,
          "action": "DELIVER",
          "delivery_id": "d1"
        },
        {
          "x": 30,
          "y": 80,
          "t": 67.08,
          "action": "DELIVER",
          "delivery_id": "d2"
        },
        {
          "x": 50,
          "y": 50,
```

```
          "t": 103.14,
          "action": "RETURN"
        }
      ]
    }
  ]
}
```

Explanation 0

The drone picks up both packages (total weight 0.7), flies to d1 (distance ≈ 22.36), delivers d1, then flies to d2 (distance ≈ 44.72), delivers d2, and returns to the warehouse (distance ≈ 36.06). Both deliveries arrive well before the deadline of 200.

- **Successful deliveries:** 2/2
- **Total energy:** 22.36×1.7 + 44.72×1.4 + 36.06×1.0 ≈ 38.01 + 62.61 + 36.06 = 136.68
- **Makespan:** 103.14
- **Raw score:** (2×100) − (136.68×0.1) − (103.14×0.05) = 200 − 13.67 − 5.16 = 181.17

Sample Input 1

```
{
  "map_size": [200, 200],
  "drones": [
    {"id": "drone_1", "max_payload": 1.0}
  ],
  "deliveries": [
    {"id": "d1", "x": 10, "y": 100, "weight": 0.3, "deadline": 200.0},
    {"id": "d2", "x": 10, "y": 10, "weight": 0.3, "deadline": 350.0},
    {"id": "d3", "x": 100, "y": 10, "weight": 0.3, "deadline": 500.0}
  ],
  "charging_stations": [
    {"x": 100, "y": 10}
  ],
  "no_fly_zones": [
    {
      "shape": "circle",
      "center": [100, 55],
      "radius": 15,
      "T_start": 0.0,
      "T_end": 150.0
    }
  ]
}
```

Sample Output 1

```
{
  "flight_manifest": [
    {
      "drone_id": "drone_1",
      "path": [
        {
          "x": 100,
          "y": 100,
          "t": 0.0,
          "action": "PICKUP",
          "delivery_ids": [
            "d1",
            "d2",
```

```
            "d3"
          ]
        },
        {
          "x": 10,
          "y": 100,
          "t": 90.0,
          "action": "DELIVER",
          "delivery_id": "d1"
        },
        {
          "x": 10,
          "y": 10,
          "t": 180.0,
          "action": "DELIVER",
          "delivery_id": "d2"
        },
        {
          "x": 100,
          "y": 10,
          "t": 270.0,
          "action": "DELIVER",
          "delivery_id": "d3"
        },
        {
          "x": 100,
          "y": 10,
          "t": 270.0,
          "action": "CHARGE"
        },
        {
          "x": 100,
          "y": 10,
          "t": 281.0,
          "action": "CHARGE_COMPLETE"
        },
        {
          "x": 100,
          "y": 100,
          "t": 371.0,
          "action": "RETURN"
        }
      ]
    }
  ]
}
```


Explanation 1

The warehouse is at (100, 100). Delivery `d3` is at (100, 10) — directly south. A circular NFZ with center (100, 55) and radius 15 sits between them on the line x=100, active from T=0 to T=150.

Why going directly to d3 first is invalid: The straight path from (100, 100) to (100, 10) passes through (100, 55) — the NFZ center. The drone would reach that point at approximately T=45, well within the NFZ's active window (T=0 to T=150). Any path segment crossing through this zone while it is active scores 0.

Route strategy — avoid the NFZ by timing: Instead of detouring around the NFZ with waypoints, the drone takes a route that naturally avoids it. It flies west to d1, south to d2, then east to d3. All three legs are far from the NFZ (closest approach is 45 units — well outside radius 15). By the time the drone needs to fly north through x=100 on the return leg (T=281), the NFZ has already expired (T=150). The drone passes through the formerly blocked area safely.

Why charging is needed: The total energy for all four legs is 522 units, which exceeds the battery capacity of 500. Conveniently, d3's location (100, 10) is also a charging station. After delivering d3 at T=270, the drone charges for 11 timesteps (22 energy at rate 2/timestep) to have enough battery (90 units) for the 90-unit return leg.

Leg-by-leg breakdown:

- Warehouse (100,100) → d1 (10,100): distance = 90, payload = 0.9, energy = 90 × 1.9 = 171. Arrives T=90 ≤ deadline 200 ✓
- d1 (10,100) → d2 (10,10): distance = 90, payload = 0.6, energy = 90 × 1.6 = 144. Arrives T=180 ≤ deadline 350 ✓
- d2 (10,10) → d3 (100,10): distance = 90, payload = 0.3, energy = 90 × 1.3 = 117. Arrives T=270 ≤ deadline 500 ✓
- Charge at (100,10): 11 timesteps, battery 68 → 90. Departs T=281.
- d3 (100,10) → Warehouse (100,100): distance = 90, payload = 0, energy = 90 × 1.0 = 90. NFZ expired at T=150, drone passes through at T≈326 ✓

Score calculation:

- **Successful deliveries:** 3/3
- **Total energy:** 171 + 144 + 117 + 90 = 522
- **Makespan:** 371
- **Raw score:** (3×100) − (522×0.1) − (371×0.05) = 300 − 52.2 − 18.55 = 229.25

This example demonstrates three key concepts: **NFZ avoidance** (the drone cannot fly south early on), **time-based NFZ expiry** (the return path is safe because the NFZ deactivated), and **charging station usage** (the drone recharges mid-trip to complete the return).